

UNIVERSITY COLLEGE LONDON

EXAMINATION FOR INTERNAL STUDENTS

MODULE CODE : **COMP0070**

ASSESSMENT PATTERN: **COMP0070A7PF 001**

MODULE NAME : **COMP0070 - Algorithmics**

LEVEL : **Postgraduate**

DATE: : **20/05/2024**

TIME : **14:30**

DURATION : **2 Hours**

This paper is suitable for candidates who attended classes for this module in the following academic year(s):

**Year
2023/24**

**EXAMINATION PAPER CANNOT BE REMOVED FROM THE EXAM HALL. PLACE
EXAM PAPER AND ALL COMPLETED SCRIPTS INSIDE THE EXAMINATION
ENVELOPE**

Hall instructions	N/A
Additional materials	N/A
Standard Calculators	Yes
Non-Standard Calculators	No

TURN OVER

Answer BOTH questions.

Marks for each part of each question are indicated in square brackets.

Standard calculators are permitted.

1. a. Let $[1,2,3,4,5,6,7,8,9,10,11,12]$ be a heap. Draw the tree representation of this heap and then give the sequence of nodes visited in (i) a preorder traversal, (ii) an inorder traversal, and (iii) a postorder traversal.

[6 marks]

- b. Give pseudocode for a binary search algorithm. The input is an array of numbers A that are sorted in order and a number x that is being searched for. The output is the position in the array where x is located if it is in the array, and where x should be inserted if it is not in the array. Briefly explain how your algorithm works.

[6 marks]

- c. Does the following algorithm work correctly with any array of numbers? If yes, briefly explain how the algorithm works using a small example. If no, briefly explain what the problem is, and give a small example to illustrate the problem.

```
Pivot( $T[i, \dots, j]$ )
 $p = T[i]$ 
 $k = i$ 
 $l = j + 1$ 
repeat  $k = k + 1$  until  $T[k] > p$ 
repeat  $l = l - 1$  until  $T[l] \leq p$ 
while  $k < l$ 
    do interchange  $T[k]$  and  $T[l]$ 
    repeat  $k = k + 1$  until  $T[k] > p$ 
    repeat  $l = l - 1$  until  $T[l] \leq p$ 
interchange  $T[i]$  and  $T[l]$ 
return  $l$ 
```

[8 marks]

[Question 1 cont. over page]

- d. Consider Prim's algorithm. Give a proof (i.e. a carefully reasoned argument) that shows that the output of the algorithm is correct. Start by specifying the input and intended output of the algorithm.

[10 marks]

- e. Let $G = (N, A)$ be a directed graph. For each node $x \in N$, let $\text{In}(x)$ be the number of arcs going into x (i.e. the **in-degree of the node**), and let $\text{Out}(x)$ be the number of arcs coming out of x (i.e. the **out-degree of the node**).

- i. If for all nodes x in N , $\text{In}(x) = \text{Out}(x)$ holds, is the graph connected? Give a carefully reasoned argument with your answer.
- ii. Does the following hold? Give a carefully reasoned argument with your answer.

$$\sum_{x \in N} \text{In}(x) = \sum_{x \in N} \text{Out}(x)$$

[8 marks]

- f. Assume a path in a graph can be represented by a set. Give pseudocode for a *recursive* algorithm that finds all the paths between a pair of nodes. The input is a graph $G = (N, A)$ and two nodes x and y . The output is the set of paths between x and y in G . Give a clear explanation of your algorithm.

[12 marks]

[Total for Question 1: 50 marks]

2. a. i. Give the definition of the mathematical *order notation* $f(n) \in O(g(n))$, and explain how this concept relates to the algorithmic idea of *worst-case analysis*. (3 marks)

- ii. Consider two algorithms A and B which solve the same problem, and have worst-case time-complexities $a(n)$ and $b(n)$ respectively in the input size n , where

$$a(n) = 3n^2 - 295n - 197$$

$$b(n) = 3n + 3$$

Which algorithm is the best asymptotically, and for which input sizes is this the case? (4 marks)

- iii. Give three possible reasons that the algorithm that is not better asymptotically might nevertheless be better in practice. (3 marks)

[10 marks]

- b. For each of the following statements, justify carefully, using the definition of big-O notation, whether they are true or false:

- i. $(n + 1)! \in O(n!)$ (3 marks);
ii. $3^n \in O(2^n)$ (3 marks);
iii. $\log_2(8n^2) \in O(\log_2(n))$ (4 marks).

[10 marks]

[Question 2 cont. over page]

- c. For each of the following short pseudocode procedures, calculate the exact number of unit-time operations each procedure requires as a function of the input size n , and then simplify your answer using O -notation.

```
i. for i = 1 to n do
    for j = 1 to 4i-3 do
        <unit cost operation>
```

(5 marks)

```
ii. for i = 1 to n do
    for j = 1 to n do
        for k = 1 to (i*j) do
            <unit cost operation>
```

(5 marks)

[10 marks]

- d. Solve the following recurrence relations and simplify your answer using O -notation. You may assume the input n is a power of 2 where appropriate.

i.

$$\begin{aligned} f(0) &= 2 \\ f(1) &= 9 \\ f(n) &= 9f(n-1) - 20f(n-2) \quad \text{for } n \geq 2 \end{aligned}$$

(5 marks)

ii.

$$\begin{aligned} f(1) &= 4 \\ f(2) &= 13 \\ f(n) &= 3f\left(\frac{n}{2}\right) - 2f\left(\frac{n}{4}\right) \quad \text{for } n > 2 \end{aligned}$$

(5 marks)

[10 marks]

[Question 2 cont. on next page]

- e. i. Define the complexity classes P, NP and Co-NP. (3 marks)
- ii. How would you go about showing that $P \neq NP$? Justify your answer. (2 marks)
- iii. How would you go about showing that $P = NP$? Justify your answer. (2 marks)
- iv. Show that if $NP \neq Co-NP$ then $P \neq NP$. (3 marks)

[10 marks]

[Total for Question 2: 50 marks]

[Total for paper: 100 marks]